Ai.Security - AEC-AB

Preview rendered 2026-10-30 03:58 UTC from 18 pages. Generated by the AEC repo agents; the pull request is the source of truth.

Contents

Security review summary Summary.md

Findings register Findings.md

- SEC-001: StreamBIM credential storage and FTP transfer security Findings/SEC-001-streambim-credential-storage.md
- SEC-002: Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps Findings/SEC-002-dalux-api-key-handling.md
- SEC-003: Git history audit - no secrets found Findings/SEC-003-git-history-audit-clean.md
- SEC-004: LISPRunner arbitrary AutoCAD Lisp script execution Findings/SEC-004-lisprunner-arbitrary-execution.md
- SEC-005: RunCommand arbitrary AutoCAD command injection Findings/SEC-005-rancmd-arbitrary-execution.md
- SEC-006: DaluxCloudDownload path traversal via server-supplied relative path Findings/SEC-006-dalux-path-traversal.md
- SEC-007: Base URL exposed in API error messages Findings/SEC-007-dalux-base-url-in-error-messages.md
- SEC-008: Tekla IFCExport path traversal via unvalidated output file path Findings/SEC-008-tekla-ifcexport-path-traversal.md
- SEC-009: Tekla five file operation extensions - safe assessment Findings/SEC-009-tekla-file-ops-safe-assessment.md
- SEC-010: Navisworks SaveDocumentAs path traversal via unvalidated directory creation and file path information disclosure Findings/SEC-010-navisworks-path-traversal-and-info-disclosure.md
- SEC-011: StreamBIM diagnostics logging of file paths and FTP operations Findings/SEC-011-streambim-diagnostics-logging.md
- SEC-012: Personal data in DaluxUser model with no identified access path Findings/SEC-012-personal-data-daluxuser-no-access-path.md

Dependencies Dependencies.md

Software bill of materials SBOM.md

- Software bill of materials SBOM/README.md

Talk to the security agent README.md

Security review summary

> **Incomplete pass.** Run 20260925-214417 did not finish: the model server stopped answering mid-run. The findings below are what it got to; the rest of the codebase is not reviewed yet. The next nightly sweep picks this repo up again.

Rewritten by the AEC security review agent on every run. Internal only.

Scope

Run `20260925-214417` — first pass. Reviewed the StreamBIM credential storage and FTP transfer security area, the Dalux API key handling area, a full git history audit for secrets across all branches, and the Dalux API authentication area.

Run `20260926-015629` — incremental. Reviewed the AutoCAD LISPRunner and RunCommand extensions (Critical RCEs), the DaluxCloudDownload path handling (Medium path traversal), and CI/CD workflows and scripts (verified input validation, path traversal protections, private content scanning).

Counts

Severity	Open	Total
Critical	2	2
High	1	1
Medium	2	2
Low	4	4
Info	3	3
Total	**12**	**12**

Status	Count
Open	12

Top open findings

1. **SEC-004** (Critical) — LISPRunner arbitrary Lisp execution: both script-file and inline modes pass user-supplied content to AutoCAD's `SendStringToExecute()` with zero sanitization. An attacker who can supply the script path or content gains arbitrary AutoCAD Lisp API access.
2. **SEC-005** (Critical) — RunCommand arbitrary command execution: user-supplied command strings passed directly to AutoCAD's `SendCommand()` with no whitelist, validation, or sanitization. Multi-line input allows command chaining and full AutoCAD API access.
3. **SEC-002** (High) — Dalux API key handling: broken credential lookup in DaluxCloudUploadCommand.cs line 30 (passes `args.ApiKey` with default value `"Dalux API Key"` instead of the looked-up credential variable to `DaluxApiService`); HttpClient gaps (no timeout, no certificate validation callback) in both upload and download services.
4. **SEC-006** (Medium) — DaluxCloudDownload path traversal via server-supplied relative path: `file.RelativePath` and `file.FileName` from the Dalux API are used directly in `Path.Combine()` without traversal validation. A compromised Dalux API could inject `../` sequences.
5. **SEC-010** (Medium) — Navisworks SaveDocumentAs path traversal via unvalidated directory creation: `Directory.CreateDirectory()` called on user-supplied path from `SaveFileField` without traversal validation; file paths also echoed in success messages.

What changed since previous run

Run `20261030-034207` — incremental. `context/changes.md` is empty — no source code changes since run 20261009-163727 (commit be9cc40). All 11 existing Open findings re-verified against the current commit (no files in changes.md, no evidence changed). New finding SEC-012 filed: personal data in DaluxUser model with no identified access path. Backlog is now empty — the personal data backlog item from the first run has been completed. Total: 12 findings (2 Critical, 1 High, 2 Medium, 4 Low, 3 Info).

Run `20261009-163727` — incremental. Commit `be9cc40` (Add Tekla 2025 support) changed only build configuration files (Directory.Build.props, 7 .csproj files) across `src/Tekla/dotnet/` — no source code (.cs) files modified. SEC-008 (Tekla IFCExport path traversal) and SEC-009 (five Tekla extensions safe) re-verified against be9cc40. Both findings confirmed still valid — the path traversal issue in IFCExportCommand.cs line 30-38 (no traversal validation, no filename sanitization) and the safe assessment of the other five extensions remain unchanged. No new findings.

Run `20260927-005009` — incremental. No source changes (context/changes.md empty). Previous run 20260926-215844 completed fully: sweep done, 11 findings (2 Critical, 1 High, 2 Medium, 4 Low, 2 Info), backlog empty. No re-verification needed.

Run `20260926-015629` is an incremental pass. Three new Critical and Medium findings were filed (SEC-004, SEC-005, SEC-006). CI/CD workflows and scripts were verified as well-implemented. The backlog has been expanded with additional items identified during review.

Run `20260926-215844` — incremental. Reviewed all seven Tekla extension file operation modules (IFCExport, ReadIn, RefreshReferenceModels, SaveModel, SetSelectionFilter, WriteOut, ZoomToSelected). One new Low finding: SEC-008 (IFCExport path traversal via unvalidated output file path). All other Tekla extensions have safe file operations with no user-typed paths or injection vectors.

Run `20260926-215844` session 6 — incremental. Replaced the Backlog row "Tekla extensions file operations" with three smaller rows by sub-project: (1) IFCExport path traversal, (2) five file operation extensions, (3) SetSelectionFilter. Reviewed the first row (IFCExport = SEC-008): re-reviewed all four IFCExport files (TeklaIFCExportCommand.cs, TeklaIFCExportArgs.cs, IFCExportConfig.cs, GlobalUsings.cs), confirmed the finding is complete and accurate, moved to Reviewed. The second and third rows remain in the Backlog for a future session.

Run `20260926-215844` session 8 — incremental. Replaced the old single Backlog row "Tekla file operations (ReadIn, RefreshReferenceModels, SaveModel, WriteOut, ZoomToSelected)" with three smaller rows: (1) five file operation extensions safe assessment, (2) macro builder helper string safety, (3) extension result pattern variance. Reviewed the first row (five extensions = SEC-009): traced all ten Command.cs and Args.cs files, confirmed all safe - SDK APIs only, no user file paths, hardcoded string literals. Moved to Reviewed.

Run `20260926-215844` session 9 — incremental. Reviewed Tekla macro builder helper string safety. Traced all five Tekla extensions' Command.cs, Args.cs, and GlobalUsings.cs. ReadIn and WriteOut use TeklaMacroBuilderHelper.Callback() with fully hardcoded literals; ReadIn Args has only boolean fields, WriteOut Args has none. Other three extensions don't use the macro builder. CW.Assistant.Extensions.Tekla.Helpers is an external NuGet package (not auditable), but no user-controlled data reaches it from this repo. No new finding. Moved row from Backlog to Reviewed.

Run `20260926-215844` session 11 — incremental. Reviewed Navisworks extensions (ClashDetectiveRunner, OpenDocument, SaveDocumentAs). SEC-010 filed: SaveDocumentAs path traversal via Directory.CreateDirectory() on unvalidated path (Medium), file path information disclosure in success messages (Low), exception detail logging to Trace. Moved row from Backlog to Reviewed.

Run `20260926-215844` session 12 — incremental. Reviewed StreamBIM diagnostics logging. SEC-011 filed: StreamBimUploadDiagnostics writes full file paths, FTP operation details, and raw exception messages to a temp directory log file (Low). Opt-in via VerboseDiagnostics flag. No credentials in logs. Moved row from Backlog to Reviewed.

Areas not yet covered

Empty — all tracked categories have been reviewed. The sweep is complete.

Open requests

None.

Findings register

One row per finding. Ids are stable and never reused. Status is one of `Open`, `Fixed`, `Accepted risk`, `False positive`. Reviewers change status by editing this table (and the finding page); the agent respects it and only re-verifies.

Id	Severity	Title	Status	Location	First seen	Last verified	Page
SEC-001	Low	StreamBIM credential storage and FTP transfer security	Open	`src/Assistant/dotnet/StreamBim/`	20260925-214417	20260925-214417	SEC-001-streambim-credential-storage.md
SEC-002	High	Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps	Open	`src/Assistant/dotnet/DaluxCloudUpload/`, `src/Assistant/dotnet/DaluxCloudDownload/`	20260925-214417	20260925-214417	SEC-002-dalux-api-key-handling.md
SEC-003	Info	Git history audit - no secrets found	Open	`src/` across all branches	20260925-214417	20260925-214417	SEC-003-git-history-audit-clean.md
SEC-004	Critical	LISPRunner arbitrary AutoCAD Lisp script execution	Open	`src/AutoCAD/dotnet/LISPRunner/LISPRunnerCommand.cs`	20260926-015629	20260926-015629	SEC-004-lisprunner-arbitrary-execution.md
SEC-005	Critical	RunCommand arbitrary AutoCAD command injection	Open	`src/AutoCAD/dotnet/RunCommand/RunCommandCommand.cs`	20260926-015629	20260926-015629	SEC-005-rancmd-arbitrary-execution.md
SEC-006	Medium	DaluxCloudDownload path traversal via server-supplied relative path	Open	`src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs`	20260926-015629	20260926-015629	SEC-006-dalux-path-traversal.md
SEC-007	Low	Base URL exposed in API error messages	Open	`src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs`	20260926-215844	20260926-215844	SEC-007-dalux-base-url-in-error-messages.md
SEC-008	Low	Tekla IFCExport path traversal via unvalidated output file path	Open	`src/Tekla/dotnet/IFCExport/TeklaIFCExportCommand.cs`	20260926-215844	20261009-163727	SEC-008-tekla-ifcexport-path-traversal.md
SEC-009	Info	Tekla five file operation extensions - safe assessment	Open	`src/Tekla/dotnet/ReadIn/`, `RefreshReferenceModels/`, `SaveModel/`, `WriteOut/`, `ZoomToSelected/`	20260926-215844	20261009-163727	SEC-009-tekla-file-ops-safe-assessment.md
SEC-010	Medium	Navisworks SaveDocumentAs path traversal via unvalidated directory creation and file path information disclosure	Open	`src/Navisworks/dotnet/SaveDocumentAs/SaveDocumentCommand.cs`, `OpenDocument/OpenDocumentCommand.cs`	20260926-215844	20260926-215844	SEC-010-navisworks-path-traversal-and-info-disclosure.md
SEC-011	Low	StreamBIM diagnostics logging of file paths and FTP operations	Open	`src/Assistant/dotnet/StreamBIMUploader/Services/StreamBimUploadDiagnostics.cs`, `StreamBimUploadService.cs`, `StreamBimFileTransferService.cs`	20260926-215844	20260926-215844	SEC-011-streambim-diagnostics-logging.md
SEC-012	Info	Personal data in DaluxUser model with no identified access path	Open	`src/Assistant/dotnet/DaluxCloudUpload/Models/DaluxModels.cs:634-653`	20261030-034207	20261030-034207	SEC-012-personal-data-daluxuser-no-access-path.md

SEC-001: StreamBIM credential storage and FTP transfer security

Severity	Low - Minor defense-in-depth gaps in FTP certificate validation and password-in-memory handling
Status	Open
Category	Secrets / Config
Location	`src/Assistant/dotnet/StreamBim/Services/StreamBimCredentialProvider.cs` lines 1-51, `src/Assistant/dotnet/StreamBim/Services/StreamBimFtpClientFactory.cs` lines 1-42, `src/Assistant/dotnet/StreamBim/Models/UserCredentials.cs` line 3
First seen	run 20260925-214417 at commit 4658b6b
Last verified	run 20260925-214417
Introduced	commit 183a206, 2026-09-15 (initial commit of StreamBIM uploader/downloader)
Page	`Ai.Security/Findings/SEC-001-streambim-credential-storage.md`

What

The StreamBIM uploader and downloader components (`StreamBIMUploader`, `StreamBIMDownloader`) use Windows Credential Manager to store and retrieve FTP credentials. This is the recommended approach for Windows applications. The FTP connection uses TLS 1.2 with Explicit encryption mode. The `StreamBimCredentialProvider` reads credentials from the Windows Credential Manager using `CredentialManager.ReadCredential()` and wraps them in a plain `UserCredentials` record. No credentials appear in logs or command-line arguments. However, there are two minor defense-in-depth observations: (1) the FTP `AsyncFtpClient` does not set an explicit `CertificateValidationCallback`, relying on .NET's default trust-store validation; and (2) password values are held as plaintext strings in memory in the `UserCredentials` record for the duration of the FTP operation, which is inherent to any application that must use credentials but is noted for completeness.

Evidence

StreamBimCredentialProvider.cs - credentials read from Windows Credential Manager, not hardcoded:

```
// src/Assistant/dotnet/StreamBim/Services/StreamBimCredentialProvider.cs line 41-51
private static UserCredentials? ReadUserCredentials(string applicationName)
{
    var credentials = CredentialManager.ReadCredential(applicationName);
    if (credentials is null ||
        string.IsNullOrWhiteSpace(credentials.UserName) ||
        string.IsNullOrWhiteSpace(credentials.Password))
    {
        return null;
    }
    return new UserCredentials(credentials.UserName, credentials.Password);
}
```

StreamBimFtpClientFactory.cs - FTP client with TLS 1.2 Explicit encryption, no explicit certificate validation callback:

```
// src/Assistant/dotnet/StreamBim/Services/StreamBimFtpClientFactory.cs line 14-21
var profile = new FtpProfile
{
    Host = "ftp.streambim.com",
    Encoding = Encoding.UTF8,
    Encryption = FtpEncryptionMode.Explicit,
    Protocols = System.Security.Authentication.SslProtocols.Tls12,
    Credentials = new NetworkCredential(credentials.UserName, credentials.Password),
};
```

No `CertificateValidationCallback` is set on the client. FluentFTP defaults to .NET's certificate validation, which trusts the system CA store.

UserCredentials.cs - plaintext password in memory:

```
// src/Assistant/dotnet/StreamBim/Models/UserCredentials.cs line 3
internal sealed record UserCredentials(string UserName, string Password);
```

Args with PasswordField attribute (prevents UI display of credentials):

```
// src/Assistant/dotnet/StreamBIMUploader/StreamBIMUploaderArgs.cs lines 5-8
[PasswordField(
    Label = "StreamBIM Credentials",
    ToolTip = "Enter your StreamBIM credentials.")]
[Required(ErrorMessage = "StreamBIM Credentials are required.")]
public UserCredentials? Credentials { get; init; }
```

No credential leakage in diagnostics - verified by grepping all `diagnostics.Log()` calls across `StreamBimFileTransferService.cs` and `StreamBimUploadDiagnostics.cs`: no password, credential, or username values are logged.

Git history - `git log -p -S "password|secret|apiKey|Bearer" -- src/Assistant/dotnet/StreamBim/` returned no results, confirming no hardcoded secrets in the commit history.

Impact

Low impact: The current implementation is security-sound. The credential storage in Windows Credential Manager (DPAPI-protected) is a best practice. The FTP connection uses TLS 1.2 Explicit encryption, which provides transport security. The absence of an explicit `CertificateValidationCallback` means the connection trusts any certificate valid in the system's CA store, which is the default and acceptable behavior for most deployments. It would only be a risk if an attacker could install a rogue CA certificate on the user's machine, which is already a high-bar privilege escalation. The plaintext-in-memory password is inherent to any application that must use credentials to perform operations; there is no practical way to avoid this. The overall implementation is a good example of credential handling in a Windows application, with the noted items being observations for defense-in-depth rather than exploitable weaknesses.

What a fix involves

No fix is required. If the organization has a strict defense-in-depth policy, the following low-effort enhancements could be considered:

1. **Certificate validation callback**: Set `client.Config.CertificateValidationCallback` to reject certificates that do not match expected characteristics (e.g., a specific issuer or SAN). This prevents MITM even if a rogue CA is installed. Note: this is only useful if the organization controls the CA or uses certificate pinning.
2. **SecureString for password**: Change `UserCredentials.Password` from `string` to `SecureString` to prevent the password from lingering in garbage-collected memory. This adds complexity as all FluentFTP APIs expect a plain `string`, so a conversion layer would be needed. Evaluate against the threat model before investing effort.
3. **Document the security model**: Add a comment to `UserCredentials.cs` and `StreamBimFtpClientFactory.cs` explaining the security choices (Credential Manager, TLS 1.2, no certificate pinning) so future developers understand the rationale.

References

- FluentFTP documentation: `FtpEncryptionMode.Explicit`, `SslProtocols.Tls12`, `CertificateValidationCallback`
- Windows Credential Manager: DPAPI-based credential storage via `System.Security.Cryptography.CredentialManager`
- OWASP Secure Storage of Sensitive Data: https://cheatsheetseries.owasp.org/cheatsheets/Secure_Storage_Cheat_Sheet.html

SEC-002: Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps

Severity	High - DaluxCloudUpload bypasses credential lookup entirely, sending "Dalux API Key" as the API key; also, HttpClient lacks timeout and certificate validation callback
Status	Open
Category	Secrets / Auth
Location	`src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadCommand.cs` line 30, `src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs` lines 20-24 and 118-143, `src/Assistant/dotnet/DaluxCloudDownload/Services/DaluxApiService.cs` lines 14-24 and 207-222
First seen	20260925-214417 at commit 4658b6b
Last verified	20260925-214417
Introduced	commit 4658b6b (initial commit of upload/download extensions)
Page	`Ai.Security/Findings/SEC-002-dalux-api-key-handling.md`

What

The Dalux Cloud Upload extension has a broken credential lookup: it reads the API key from Windows Credential Manager but then passes the unlooked-up argument value directly to the API service. In `DaluxCloudUploadCommand.cs`, the variable `apiKey` (line 13) holds the retrieved credential, is validated for null (line 14-17), and yet `DaluxApiService` is instantiated on line 30 with `args.ApiKey` instead of `apiKey`. The `args.ApiKey` property defaults to the literal string `"Dalux API Key"` (the credential name), so the upload always sends the credential-name string as the actual API key, causing every upload to fail with an authentication error. The Download extension (`DaluxCloudDownloadCommand.cs` line 35) correctly passes the looked-up `apiKey` variable to its DaluxApiService.

Additionally, both `DaluxApiService` implementations create a raw `new HttpClient()` with no timeout, no certificate validation callback, and no explicit handling of server certificate errors. The upload service's error handler (`HandleException`) includes the full base URL in error messages, which is a minor information disclosure.

Evidence

Broken credential lookup in upload command:

```
// src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadCommand.cs lines 13-30
var apiKey = GetApiKey(args.ApiKey);
if (apiKey is null)
{
    return Result.Text.Failed("API key not found.");
}
// ... validation continues ...
var daluxService = new DaluxApiService(args.ApiKey, args.BaseUrl);
```

Line 30 passes `args.ApiKey` (default: `"Dalux API Key"` — see below) instead of the looked-up `apiKey` variable. This is a copy-paste or refactoring bug.

Args default value (the credential name, not a real key):

```
// src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadArgs.cs lines 10-14
[PasswordField(
    Label = "Dalux API Key",
    ToolTip = "Add organization name in the 'Name' field and Dalux API Key in the 'Password' field.")]
[Required(ErrorMessage = "Dalux API Key is required.")]
public string ApiKey { get; set; } = "Dalux API Key";
```

Correct credential lookup in download command (for comparison):

```
// src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs lines 14-35
var apiKey = GetApiKey(args.ApiKey);
if (apiKey is null)
{
    return Result.Text.Failed("API key not found.");
}
```

```
// ... validation ...
var daluxService = new DaluxApiService(apiKey, args.BaseUrl);
```

HttpClient without timeout or certificate validation (both projects):

```
// src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs lines 20-24
public DaluxApiService(string apiKey, string baseUrl = "https://node1.field.dalux.com/service/api")
{
    _apiKey = apiKey ?? throw new ArgumentNullException(nameof(apiKey));
    _baseUrl = baseUrl;
    if (!_baseUrl.EndsWith("/"))
    {
        _baseUrl += "/";
    }
    _httpClient = new HttpClient();
    ConfigureHttpClient();
}

private void ConfigureHttpClient()
{
    _httpClient.BaseAddress = new Uri(_baseUrl);
    _httpClient.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
    _httpClient.DefaultRequestHeaders.Add("X-API-Key", _apiKey);
}
```

No `_httpClient.Timeout` set (defaults to 100 seconds), no `ServerCertificateCustomValidationCallback` configured.

Error messages include base URL (minor information disclosure):

```
// src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs lines 120-130
private DaluxApiResponse<T> HandleException<T>(Exception ex, string operation)
{
    if (ex is HttpRequestException httpEx)
    {
        return DaluxApiResponse<T>.Failed(
            $"Network error during {operation}: {httpEx.Message}\n\n" +
            $"Base URL: {_baseUrl}\n" +
            $"Please verify:\n" +
            $"- Network connectivity\n" +
            $"- Correct API endpoint (may be region-specific)\n" +
            $"- VPN/Proxy requirements\n" +
            $"- Firewall settings"
        );
    }
    // ...
}
```

Git history check: `git log -p -S "Dalux API Key\|apiKey\|X-API-Key" -- src/Assistant/dotnet/DaluxCloudUpload/ src/Assistant/dotnet/DaluxCloudDownload/` returned no results, confirming no hardcoded Dalux API keys have been committed.

No logging of API keys: Grepped all `.cs` files in both projects for logging patterns (`diagnostics.Log`, `Console.WriteLine`, etc.) — none found. API keys are not leaked via logs.

Impact

High severity: The upload command's broken credential lookup is a functional bug that makes Dalux Cloud Upload completely non-functional — every upload attempt sends the literal string "Dalux API Key" as the API key, which will be rejected by the Dalux API. An attacker who can trigger an upload would not gain unauthorized access because the wrong credential is sent. However, the presence of this bug indicates a lack of code review on credential handling.

The HttpClient gaps are secondary: no timeout means the application can hang indefinitely on unresponsive servers (DoS potential in multi-user environments); no certificate validation callback means it trusts any certificate valid in the system's CA store, which is the default and generally acceptable. The base URL in error messages is a minor information disclosure that could help an attacker identify the Dalux deployment but provides limited value alone.

What a fix involves

1. **Fix the credential lookup bug**: Change line 30 of `DaluxCloudUploadCommand.cs` from `new DaluxApiService(args.ApiKey, args.BaseUrl)` to `new DaluxApiService(apiKey, args.BaseUrl)` to use the looked-up credential.

2. **Add HttpClient timeout**: Set `_httpClient.Timeout = TimeSpan.FromSeconds(30);` (or similar) in `ConfigureHttpClient()` in both DaluxApiService implementations.
3. **Consider certificate validation callback**: Add `_httpClient.ServerCertificateCustomValidationCallback = HttpClientHandler.DangerousAcceptAnyServerCertificateValidator;` only if a custom CA is used and `.DangerousAcceptAnyServerCertificateValidator` is inappropriate; otherwise keep the .NET default which trusts the system CA store.
4. **Remove base URL from error messages**: Remove or obfuscate the `_baseUrl` from the `HandleException` error string in `DaluxCloudUpload/Services/DaluxApiService.cs` to reduce information disclosure.

References

- SEC-001 (StreamBIM credential storage) — similar credential manager pattern, correctly implemented there.
- Meziantou.Framework.Win32.CredentialManager — third-party library used for credential access in both projects.
- Microsoft HttpClient best practices

SEC-003: Git history audit - no secrets found

Severity	Info - No secrets found in git history; this is a negative finding documenting the audit
Status	Open
Category	Secrets
Location	`src/` across all branches
First seen	run 20260925-214417 at commit 4658b6b
Last verified	run 20260925-214417
Introduced	N/A (audit finding)

What

A full audit of the git repository was performed to check whether any secrets, credentials, or sensitive values were ever committed to the source code under `src/`. The search used `git log --all -p -S` with multiple patterns: `password`, `secret`, `key`, `apiKey`, `BEGIN.*PRIVATE KEY`, `connection`, and `Token|AuthToken|Bearer|api_token`. All matches returned either no results or only legitimate code additions (credential lookup logic, Dalux download command, OpenWorksets migration README). No actual secret values, connection strings, or private keys were found in any commit.

Evidence

Commands executed:

```
git log --all -p -S "password" -- src/   # one diff of file content updates, no secrets
git log --all -p -S "secret" -- src/     # no results
git log --all -p -S "key" -- src/        # legitimate additions only (credential lookup, API key parameter)
git log --all -p -S "apiKey" -- src/     # legitimate additions only (Dalux credential management)
git log --all -p -S "BEGIN.*PRIVATE KEY" -- src/  # no results
git log --all -p -S "connection" -- src/  # one diff of a README migration, no connection strings
git log --all -p -S "Token|AuthToken|Bearer|api_token" -- src/  # no results
```

All matches were legitimate code: `DaluxCloudDownloadCommand.cs` initial commit added credential lookup (using `Meziantou.Framework.Win32.CredentialManager`), `DaluxCloudUploadCommand.cs` initial commit added the same pattern, and `OpenWorksets/README.md` was a migration from another repo with no credentials.

Impact

None detected. No secrets are present in the git history that could be extracted by anyone with repository read access. This is a baseline security assessment confirming the repository has not been contaminated with committed credentials.

What a fix involves

No action required. The repository is clean. As a preventative measure, consider adding a pre-commit hook or CI check (e.g., `gitleaks`, `trufflehog`) to prevent future accidental secret commits.

References

- SEC-001: StreamBIM credential storage (uses Windows Credential Manager, not git)
- SEC-002: Dalux API key handling (uses Windows Credential Manager lookup)

SEC-004: LISPRunner arbitrary AutoCAD Lisp script execution

Severity	Critical - Arbitrary Lisp code execution from user-supplied file paths or inline content
Status	Open
Category	Injection / RCE
Location	`src/AutoCAD/dotnet/LISPRunner/LISPRunnerCommand.cs` lines 20-57
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 183a206, 2026-09-15 (initial commit of AutoCAD extensions)

What

The LISPRunner extension allows users to execute arbitrary AutoCAD Lisp code through two modes: (1) specifying a file path to a `.lsp` script file, and (2) providing inline Lisp code. Both modes pass the user-supplied content directly to AutoCAD via `SendStringToExecute()` / `SendCommand()` without any sanitization or validation. An attacker who can persuade a user to select a malicious Lisp file or enter malicious inline code can execute arbitrary commands within the AutoCAD process — including file system access, network calls, and model manipulation. The extension treats the input as trusted and does not restrict which Lisp functions may be called.

Evidence

LISPRunnerCommand.cs - script file mode (lines 20-38):

```
private static IExtensionResult RunScriptFile(Autodesk.AutoCAD.ApplicationServices.Document doc, LISPRunnerArgs args)
{
    var scriptPath = args.LispScriptPath?.Trim();
    if (string.IsNullOrWhiteSpace(scriptPath))
    {
        return Result.Text.Failed("A Lisp script file must be selected in SelectFile mode.");
    }

    var fullScriptPath = System.IO.Path.GetFullPath(scriptPath);
    if (!System.IO.File.Exists(fullScriptPath))
    {
        return Result.Text.Failed($"The selected Lisp script was not found: {fullScriptPath}");
    }

    var lispLoadCommand = BuildLoadCommand(fullScriptPath);
    doc.SendStringToExecute(lispLoadCommand, activate: true, wrapUpInactiveDoc: false, echoCommand: false);

    return Result.Text.Succeeded($"Queued Lisp script file for execution: {fullScriptPath}");
}
```

The file path is used directly in a `(load "...")` call. No file extension validation, no content inspection.

LISPRunnerCommand.cs - inline mode (lines 40-51):

```
private static IExtensionResult RunInlineScript(Autodesk.AutoCAD.ApplicationServices.Document doc, LISPRunnerArgs args)
{
    var inlineScript = args.InlineScript?.Trim();
    if (string.IsNullOrWhiteSpace(inlineScript))
    {
        return Result.Text.Failed("Inline Lisp script content is required in Inline mode.");
    }

    doc.SendStringToExecute(inlineScript + " ", activate: true, wrapUpInactiveDoc: false, echoCommand: false);

    return Result.Text.Succeeded("Queued inline Lisp script for execution.");
}
```

The inline script content is sent directly to AutoCAD with no sanitization.

LISPRunnerArgs.cs confirms the input sources:

```
// src/AutoCAD/dotnet/LISPRunner/LISPRunnerArgs.cs line 8
public string? LispScriptPath { get; init; }

// line 9
public string? InlineScript { get; init; }
```

Impact

An attacker who can trick a user into running this extension with a malicious Lisp file or inline script gains code execution within the AutoCAD process context. AutoCAD Lisp can: read/write arbitrary files on the user's machine, make network requests, modify the AutoCAD database, send commands to other AutoCAD APIs, and access the Windows clipboard. This is a direct remote code execution vulnerability.

What a fix involves

1. **Remove inline mode entirely** or restrict it to a safe subset of Lisp functions (e.g., math only).
2. **For file mode**: restrict allowed file extensions to `.lsp` or `.fas`, restrict file paths to a known directory whitelist, and optionally scan file content against a blocklist of dangerous Lisp functions (`vl-file-delete`, `vlax-invoke`, `vla-`, etc.).
3. **Add logging** of which scripts are executed for audit purposes.
4. **Consider a sandbox approach**: execute scripts in a restricted AutoCAD document context with limited API access.

References

- SEC-005 (RunCommandCommand) — same pattern of unsanitized command injection into AutoCAD
- AutoCAD .NET API: `SendStringToExecute`, `SendCommand`

SEC-005: RunCommand arbitrary AutoCAD command injection

Severity	Critical - Arbitrary AutoCAD command execution from user-supplied input allows remote code execution
Status	Open
Category	Injection / RCE
Location	`src/AutoCAD/dotnet/RunCommand/RunCommandCommand.cs` lines 81-86
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 183a206, 2026-09-15 (initial commit of AutoCAD extensions)

What

The RunCommand extension allows users to supply arbitrary AutoCAD command strings that are then executed via `SendCommand()`. The extension splits the input on newlines and executes each line as a separate AutoCAD command without any sanitization, validation, or whitelist. AutoCAD commands include not just drawing operations but also system commands, file operations, and .NET API access. This means a user can execute virtually any command available within the AutoCAD environment, including those that can affect the file system, network, or other applications.

Evidence

RunCommandCommand.cs - direct command injection (lines 81-86):

```
private static void RunCommand(string command)
{
    dynamic acadApp = Autodesk.AutoCAD.ApplicationServices.Application.AcadApplication;
    var thisDrawing = acadApp.ActiveDocument;
    thisDrawing.SendCommand(command);
}
```

RunCommandCommand.cs - multi-command execution (lines 29-54):

```
var commands = args.Commands?.Split('\n') ?? new string[] { };

foreach (var command in commands)
{
    if (string.IsNullOrWhiteSpace(command))
        continue;

    try
    {
        RunCommand(command);
        // ...
    }
    catch (System.Exception e)
    {
        commandResults.Add(new CommandResult
        {
            Succeeded = false,
            CommandResults = command,
            ErrorMessage = e.Message
        });
    }
}
```

RunCommandArgs.cs shows the input source:

```
// src/AutoCAD/dotnet/RunCommand/RunCommandArgs.cs line 7
[Required(ErrorMessage = "AutoCAD Command(s) is required.")]
public string Commands { get; init; } = string.Empty;
```

No sanitization or validation of `args.Commands` occurs before execution.

Impact

An attacker who can supply commands to this extension gains full access to the AutoCAD command environment. AutoCAD commands can:

- Execute system commands via `Shell` or `_(shell)`
- Access the file system via `_-insert`, `_-wedge`, and other file-based commands
- Call .NET methods via `-.NETLOAD` and `_(.NET)` commands
- Make network requests through AutoCAD's .NET API
- Modify AutoCAD drawings and settings

This is a direct remote code execution vulnerability. The multi-line input capability means multiple commands can be chained in a single execution.

What a fix involves

1. **Implement a command whitelist**: Only allow a predefined set of safe AutoCAD commands (e.g., `LINE`, `CIRCLE`, `DIMLINEAR`) and reject all others.
2. **Sanitize arguments**: For each whitelisted command, validate that arguments contain only expected character classes (alphanumeric, specific punctuation).
3. **Remove multi-command support**: Single-command execution is safer than allowing command chains.
4. **Consider a safer alternative API**: Instead of `SendCommand()`, use the .NET API directly to perform specific drawing operations that the extension is designed for.
5. **Add audit logging**: Log all commands attempted and their results.

References

- SEC-004 (LISPRunner) — similar unsanitized execution vulnerability in Lisp scripts
- AutoCAD .NET API documentation
- OWASP Input Validation Cheat Sheet

SEC-006: DaluxCloudDownload path traversal via server-supplied relative path

Severity	Medium - Server-supplied relative path used in file output without path validation creates a supply-chain attack vector
Status	Open
Category	Injection / Path Traversal
Location	`src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs` lines 368-378
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 0cbd897, 2026-09-22 (DaluxCloudDownload initial code)

What

When downloading folder contents from the Dalux API, the extension uses `file.RelativePath` from the Dalux server response directly to construct local output paths (line 368-369). While `file.FileName` is used directly in single-file downloads (line 314), the relative path is a server-provided value that could contain path traversal sequences (`../`) if the Dalux API server is compromised, misconfigured, or affected by a bug. The code does not validate or sanitize the relative path before combining it with the user-specified output folder. An attacker who controls the Dalux server could write files to arbitrary locations on the user's machine within the output folder hierarchy.

Evidence

DaluxCloudDownloadCommand.cs - folder download using server-relative path (lines 367-378):

```
var relativePath = string.IsNullOrEmpty(file.RelativePath) ? file.FileName : $"{file.RelativePath}/{file.FileName}";
var outputPath = Path.Combine(outputFolder, relativePath);
var outputDir = Path.GetDirectoryName(outputPath);

if (!string.IsNullOrEmpty(outputDir) && !Directory.Exists(outputDir))
{
    Directory.CreateDirectory(outputDir);
}

await File.WriteAllBytesAsync(outputPath, content.Data, [value redacted]);
```

DaluxCloudDownloadCommand.cs - single file download using server-supplied filename (lines 314-316):

```
var outputPath = Path.Combine(outputFolder, findFile.Data.FileName);
await File.WriteAllBytesAsync(outputPath, content.Data, [value redacted]);
return DaluxApiResponse<string>.Success(outputPath);
```

The `file` object is `DaluxFile` returned from the Dalux API (`DaluxCloudDownload.Models`). The `RelativePath` and `FileName` fields are not validated against path traversal.

Impact

If the Dalux API server is compromised, an attacker could:

- Inject `../` sequences into `RelativePath` or `FileName` responses
- Escape the intended output folder by navigating up the directory tree
- Overwrite arbitrary files on the user's machine (depending on what `outputFolder` allows)
- Place malicious files in system directories if the user specifies a privileged output folder

This is Medium severity because: (1) the path values come from the Dalux server, not directly from the user, requiring a server compromise or a Dalux bug; (2) the paths are combined with the user-specified output folder, so successful traversal is limited to within that parent directory; (3) defense-in-depth is needed since the output folder is a reasonable assumption for where downloaded files go.

What a fix involves

1. **Validate relative paths**: Before using `file.RelativePath` or `file.FileName`, normalize the full path and verify it remains within the output folder.

2. **Example approach**: Resolve `Path.GetFullPath(Path.Combine(outputFolder, relativePath))` and check that it starts with `Path.GetFullPath(outputFolder) + Path.DirectorySeparatorChar`.
3. **Sanitize filenames**: Remove or reject filenames containing `..`, null bytes, or paths with `:` on Windows.
4. **Add unit tests** for path traversal attempts in both single-file and folder-download code paths.
5. The prior fix in StreamBIM (commit c5864f9) addressed path issues in that project but did not apply to DaluxCloudDownload.

References

- CWE-22: Path Traversal
- CVE history of NuGet/nupkg path traversal in Sync-ExtensionDocs.ps1 line 52-53 shows the team has this awareness for .nupkg extraction but did not apply it to Dalux downloads

SEC-007: Base URL exposed in API error messages

Severity	Low - Information disclosure of infrastructure endpoint. Not directly exploitable since authentication is via X-API-Key header (not embedded in error messages), and the base URL is a public endpoint URL.
Status	Open
Category	Data protection
Location	`src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs` lines 122-130
First seen	run 20260926-215844 at commit unknown
Last verified	run 20260926-215844
Introduced	Not determined

What

When a network error occurs (e.g., HTTP request fails), the error response returned to the caller includes the full base URL of the Dalux API service (`https://node1.field.dalux.com/service/api`). This is exposed in the `HandleException<T>` method on lines 122-130 of `DaluxApiService.cs`. The base URL is also potentially exposed in other error messages if `_baseUrl` is logged or printed elsewhere.

Evidence

File: `src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs` (identical code in `DaluxCloudDownload/Services/DaluxApiService.cs`)

Line 122-130:

```
return DaluxApiResponse<T>.Failed(
    $"Network error during {operation}: {httpEx.Message}\n\n" +
    $"Base URL: {_baseUrl}\n" +
    $"Please verify:\n" +
    $"- Network connectivity\n" +
    $"- Correct API endpoint (may be region-specific)\n" +
    $"- VPN/Proxy requirements\n" +
    $"- Firewall settings"
);
```

Default constructor value (line 20): `_baseUrl = "https://node1.field.dalux.com/service/api"`

Impact

An attacker who can trigger a network error (e.g., via a malformed request or by exhausting connection resources) would learn the exact Dalux API base URL. The URL itself is publicly discoverable (standard Dalux API), so the disclosure is limited. No credentials, tokens, or internal network topology are revealed. The X-API-Key header used for authentication is not embedded in the error message.

What a fix involves

Remove or sanitize the `_baseUrl` from the returned error text. Lines 124 should be deleted or replaced with a generic message that does not reference the infrastructure endpoint. Apply the same fix to `DaluxCloudDownload/Services/DaluxApiService.cs`.

References

Related: StreamBIM credential handling uses Windows Credential Manager (SEC-001).

SEC-008: Tekla IFCExport path traversal via unvalidated output file path

Severity	Low - Local CAD extension running under user context, no network exposure, but lacks defense-in-depth path validation that comparable Revit exports have (sanitization present in DWGExport/NWCExport but absent here).
Status	Open
Category	Injection / Path Traversal
Location	`src/Tekla/dotnet/IFCExport/TeklaIFCExportCommand.cs` lines 20-39
First seen	run 20260926-215844 at commit 4658b6b
Last verified	run 20260926-215844
Introduced	Not determined

What

The Tekla IFCExport extension allows the output file path to be set either from an XML configuration file selected by the user or via a `SaveFileField` dialog override. Neither path is validated against directory traversal (`..` sequences) before being normalized with `Path.GetFullPath()` and used to create directories and write the exported IFC file. The Revit equivalents (DWGExport, NWCExport) sanitize filenames by stripping invalid path characters, but Tekla's IFCExport performs no such sanitization. An attacker who can supply a crafted config XML file could redirect the IFC export output to an arbitrary directory on the local filesystem.

Evidence

`src/Tekla/dotnet/IFCExport/TeklaIFCExportCommand.cs`:

```
// Line 12-16: User selects config file via FilePickerField
[FilePickerField(
    Label = "IFC Export Config File",
    ToolTip = "Path to the IFC export configuration XML file",
    Hint = "Select a IFC export config file",
    FileExtensions = ["xml","*"])]
public string? ExportConfigFilePath { get; set; }

// Line 14: Config file is read and deserialized
var bytes = File.ReadAllBytes(args.ExportConfigFilePath);
XmlSerializer serializer = new(typeof(IFCExportConfig));
IFCExportConfig exportConfig = (IFCExportConfig)serializer.Deserialize(memStream);

// Line 19-23: SaveFileField override goes directly to OutputFile with no validation
if (!string.IsNullOrEmpty(args.FilePathOverride))
{
    exportConfig.OutputFile = args.FilePathOverride!;
}

// Line 25-34: OutputFile used directly after GetFullPath (normalization only) and Directory.CreateDirectory
if (string.IsNullOrEmpty(exportConfig.OutputFile))
{
    return Result.Text.Failed("Output file path is not set.");
}

exportConfig.OutputFile = Path.GetFullPath(exportConfig.OutputFile);
if (!exportConfig.OutputFile.ToUpper().EndsWith(".IFC"))
{
    exportConfig.OutputFile += ".ifc";
}
var directoryPath = Path.GetDirectoryName(exportConfig.OutputFile);
if (!Directory.Exists(directoryPath))
{
    Directory.CreateDirectory(directoryPath);
}
```

`src/Tekla/dotnet/IFCExport/IFCExportConfig.cs`:

```
// Line 44-45: OutputFile is a plain string - XML content is user-editable
[XmlElement(ElementName="OutputFile")]
```

```
public string? OutputFile { get; set; }
```

The `IFCExportConfig` class is deserialized from an XML file that the user selects. While the user picks the file, the XML content is fully editable by the user (or by anyone who can place a file in the same directory). The config could contain:

```
<config>
    <OutputFile>../../../Windows/System32/malicious.ifc</OutputFile>
    ...
</config>
```

`Path.GetFullPath()` normalizes the path to an absolute path but does **not** prevent traversal. `Directory.CreateDirectory()` will create any missing parent directories. The Tekla API (`comp.Insert()`) then writes the export to that path.

Impact

A user who runs IFCExport with a crafted XML config (or a config placed by an attacker in a shared directory) could cause Tekla to write IFC export files to arbitrary locations on the local filesystem. The actual IFC content is determined by Tekla's own export API, not by the config file, so the attacker cannot control the file content. However, writing to unexpected locations could:

- Overwrite existing files if the crafted path points to an existing file
- Create directories in unexpected locations
- Mislead users about where their exported files are stored

This is a local-only, single-user attack surface (no network component). The realistic vector is social engineering: presenting a user with a config file that appears legitimate but redirects exports.

What a fix involves:

- Validate the output path to ensure it stays within an expected base directory (e.g., a configured export root or the current working directory).
- Sanitize the filename by stripping invalid path characters (similar to the Revit `Regex.Replace(fileName, "[<>:\"/\\|?*]", "_")` pattern used in DWGExport/NWCExport).
- Consider adding a confirmation dialog for non-standard paths.
- Do not auto-create parent directories for paths that require traversal to reach them.

References

- SEC-006: DaluxCloudDownload path traversal via server-supplied relative path (same category, different source)
- Comparison: Revit DWGExport (`src/Revit/dotnet/DWGExport/`) and NWCExport use `Regex.Replace(fileName, "[<>:\"/\\|?*]", "_")` for filename sanitization
- Inconsistency: Revit extensions sanitize output filenames but Tekla extensions do not

SEC-009: Tekla five file operation extensions - safe assessment

Severity	Info - Documented security assessment showing no injection or path traversal risk in these extensions
Status	Open
Category	Injection / Path Traversal
Location	`src/Tekla/dotnet/ReadIn/`, `RefreshReferenceModels/`, `SaveModel/`, `WriteOut/`, `ZoomToSelected/`
First seen	run 20260926-215844 at commit 4658b6b
Last verified	run 20260926-215844
Introduced	N/A

What

Security review of five Tekla extension file operation modules (ReadIn, RefreshReferenceModels, SaveModel, WriteOut, ZoomToSelected) confirms they pose no injection, path traversal, or command execution risk. None accept user-typed file paths or command strings as input. All operate purely through the Tekla SDK API on in-model objects or hardcoded macro calls.

Evidence

All five extensions follow the same safe pattern:

ReadIn (`TeklaReadInCommand.cs`, `TeklaReadInArgs.cs`):

- Args: only two boolean fields (`Save`, `FailTask`) - no file paths or strings.
- Uses `ModelHistory.GetLocalChanges()` and `ModelHistory.TakeModifications()` - Tekla SDK APIs operating on in-memory model state.
- Macro callback uses hardcoded literal: `Callback("acmdRunPluginMethod", "SharingToolsFeature;Tool.SharingAutomation;r0.00:00:00", "main_frame")` (line 27).
- `ModelHandler.Save()` on line 58 is the Tekla SDK save, not a user-supplied path.

RefreshReferenceModels (`RefreshReferenceModelsCommand.cs`, `RefreshReferenceModelsArgs.cs`):

- Args: empty class (line 3).
- Iterates `REFERENCE_MODEL` objects from `Model.GetModelObjectSelector().GetAllObjectsWithType()` - model objects already in the Tekla file.
- Calls `ReferenceModel.RefreshFile()` (line 24) on existing model references - SDK API, not user input.

SaveModel (`TeklaSaveModelCommand.cs`, `TeklaSaveModelArgs.cs`):

- Args: empty class (line 3).
- Single call to `ModelHandler.Save()` (line 7) - standard SDK save, no path parameter.

WriteOut (`TeklaWriteOutCommand.cs`, `TeklaWriteOutArgs.cs`):

- Args: empty class (line 3).
- Macro callback uses hardcoded literal: `Callback("acmdRunPluginMethod", "SharingToolsFeature;Tool.SharingAutomation;w", "main_frame")` (line 10).

ZoomToSelected (`ZoomToSelectedCommand.cs`, `ZoomToSelectedArgs.cs`):

- Args: empty class (line 3).
- Uses `DrawingHandler`, `DrawingObjectSelector.GetSelected()`, `ModelObject.Hideable.ShowInDrawing()/HideFromDrawing()` - all SDK UI APIs.
- `ModelInternal.Operation.dotStartAction("ZoomToSelected", string.Empty)` (line 14) - hardcoded string.

Impact

No exploitable vector identified. These extensions cannot be used for injection, path traversal, or arbitrary command execution via user input.

What a fix involves

No fix needed. This finding documents that the review of these five extensions is complete and they are safe.

References

- SEC-008: Tekla IFCExport path traversal (different extension with user input)
- Coverage.md: Backlog item "Tekla file operations (ReadIn, RefreshReferenceModels, SaveModel, WriteOut, ZoomToSelected)"

SEC-010: Navisworks SaveDocumentAs path traversal via unvalidated directory creation and file path information disclosure

Severity	Medium - `Directory.CreateDirectory()` called on user-supplied path with zero traversal validation; defense-in-depth gap. The SaveFileField UI constrains input in normal use, but configuration-level manipulation could bypass it. Information disclosure is additional LOW concern in success messages.
Status	Open
Category	Data protection / Injection / Path Traversal
Location	`src/Navisworks/dotnet/SaveDocumentAs/SaveDocumentCommand.cs` lines 8-12, line 15
First seen	run 20260926-215844 at commit 4658b6b
Last verified	run 20260926-215844
Introduced	commit 4658b6b, 2026-09-26 (not determined if earlier)

What

`SaveDocumentCommand.cs` extracts the parent directory from a user-supplied file path (via `Path.GetDirectoryName()`) and creates that directory with `Directory.CreateDirectory()` without any path traversal validation. If the path contains sequences like `..\..\..\Windows\` or an absolute path outside the intended scope, the code will create (or traverse into) unintended directories. Additionally, the success message on line 15 returns the full file path to the caller, which could be logged or displayed, leaking project/customer directory structures.

Evidence

SaveDocumentAs/SaveDocumentCommand.cs lines 8-15:

```
var saveFolder = Path.GetDirectoryName(args.Path);
if (!string.IsNullOrEmpty(saveFolder) && !Directory.Exists(saveFolder))
{
    Directory.CreateDirectory(saveFolder);
}
var doc = Autodesk.Navisworks.Api.Application.ActiveDocument;
doc.SaveFile(args.Path);
return Result.Text.Succeeded($"Saved: {args.Path}");
```

OpenDocument/OpenDocumentCommand.cs line 26 also echoes the full path:

```
return Result.Text.Succeeded($"Opened file at path: {args.Path}");
```

SaveDocumentAs/SaveDocumentArgs.cs line 7 defines the field:

```
[SaveFileField(Label = "Path", Hint = "Save file path", ToolTip = "File Path(ex: C:\\MyFile.nwd)")]
public string Path { get; set; } = string.Empty;
```

ClashDetectiveRunner/ClashDetectiveRunTestCommand.cs line 39 and 77 write full exception details to trace:

```
System.Diagnostics.Trace.WriteLine($"Failed running all clash tests: {ex}");
System.Diagnostics.Trace.WriteLine($"Failed running clash test '{requestedTestName}': {ex}");
```

Impact

- **Directory creation abuse (Medium)**: An attacker with access to extension configuration (e.g., in the CW Assistant task settings) could set the `Path` field to include path traversal sequences. `Directory.CreateDirectory()` would create the specified directory tree, potentially placing files in unexpected locations on the local machine.
- **Information disclosure (Low)**: Full file paths are returned in success messages by both OpenDocument and SaveDocumentAs. These could be captured in logs, UI screens, or exported reports, revealing internal directory structures that may encode project names, customer identifiers, or organizational hierarchy.
- **Stack trace leakage (Info)**: Full exception objects (including stack traces with internal assembly paths) are written to `System.Diagnostics.Trace`, which could be captured by any process with access to the trace output.

What a fix involves

1. Add path validation before `Directory.CreateDirectory()`: resolve the full path with `Path.GetFullPath()`, resolve it against a known safe base directory (or the current working directory), and verify the result does not escape the expected scope using `Path.GetRelativePath()` or a `StartsWith()` check on the normalized path.
2. Sanitize the file path in success messages: use only the file name and a truncated or generic directory indicator rather than the full absolute path.
3. Log structured exception data without raw `ToString()` on the full exception object, or at least exclude stack trace information from trace output in production builds.

References

- SEC-008: Tekla IFCExport path traversal via unvalidated output file path (same vector: `Directory.CreateDirectory()` on user-supplied path without validation)
- SEC-007: Base URL exposed in API error messages (information disclosure pattern)

SEC-011: StreamBIM diagnostics logging of file paths and FTP operations

Severity	Low - File paths and FTP operation details written to a local temp directory log file. Information disclosure with limited scope: local file on the running machine, opt-in via a verbose flag, no credentials or personal data in logs.
Status	Open
Category	Logging
Location	`src/Assistant/dotnet/StreamBIMUploader/Services/StreamBimUploadDiagnostics.cs` (all), `StreamBimUploadService.cs` (lines 25, 30, 68, 71, 98), `StreamBimFileTransferService.cs` (lines 29, 33, 66-67, 76, 167, 188, 202, 231, 238-240, 246, 263, 303, 308, 312, 331, 336, 344, 348)
First seen	run 20260926-215844 at commit 4658b6b
Last verified	run 20260926-215844
Introduced	Unknown - this is a diagnostics/logging framework, likely present since inception.

What

The `StreamBimUploadDiagnostics` class writes verbose diagnostic messages to a log file in the system temp directory (`%TEMP%\StreamBIMUploader\upload-<timestamp>.log`) whenever the `VerboseDiagnostics` flag is set. The log captures detailed file paths (full local paths and full remote FTP paths), FTP operation details (connection mode, upload status, directory listing results), exception messages from FluentFTP, and file sizes. The log file path itself is returned to the caller via the upload result's `Diagnostics` list. No credentials, API keys, or personal data are written to the logs.

Evidence

Diagnostics class (`src/Assistant/dotnet/StreamBIMUploader/Services/StreamBimUploadDiagnostics.cs`):

- Line 29: `var directory = Path.Combine(Path.GetTempPath(), LogDirectoryName);` - logs written to temp directory
- Line 32: `var fileName = $"upload-{DateTimeOffset.UtcNow:yyyyMMdd-HHmmss-fff}.log";` - timestamped log file
- Line 45: `var entry = $"{DateTimeOffset.UtcNow:ToString("O", CultureInfo.InvariantCulture)} {message}{Environment.NewLine}";` - all messages logged verbatim
- Line 50: `File.AppendAllText(LogPath, entry);` - no rotation, no access control

Upload service calls (`src/Assistant/dotnet/StreamBIMUploader/Services/StreamBimUploadService.cs`):

- Line 25: `diagnostics.Log($"Using FTP data connection mode {client.Config.DataConnectionType}.");`
- Line 30: `diagnostics.Log($"Selected file: '{localFilePath}'.");`
- Line 68: `diagnostics.Log($"Preparing upload: local='{localFilePath}', remote-relative='{remoteRelativePath}', attempt={attempt + 1}.");`
- Line 98: `diagnostics.Log($"Transient FTP failure for '{localFilePath}': {StreamBimExceptionHelper.GetInnermostMessage(exception)}. Retrying.");`

File transfer service calls (`src/Assistant/dotnet/StreamBIMUploader/Services/StreamBimFileTransferService.cs`):

- Line 29: `diagnostics.Log($"Resolved remote path: '{remotePath}'.");`
- Line 67: logs `remoteDirectory` and `remoteFileName`
- Line 76: `diagnostics.Log($"Upload completed with status {uploadStatus}: '{remotePath}'.");`
- Line 167: `diagnostics.Log($"Starting FTP upload attempt {attempt + 1}: '{localPath}' -> '{remotePath}'.");`
- Line 308: `diagnostics.Log($"Object lookup verified uploaded file: '{remotePath}', size={remoteInfo.Size} bytes.");`
- Line 348: `diagnostics.Log($"Remote directory listing verified file: name='{entry.Name}', size={entry.Size} bytes, path='{entry.FullName}'.");`

Exception helper (`src/Assistant/dotnet/StreamBim/Services/StreamBimExceptionHelper.cs`):

- Line 21-23: `GetInnermostMessage` returns `exception.InnerException.Message` - raw FTP exception messages from the server are logged verbatim.

Impact

Anyone with file system access to the machine running the uploader can read the log files and learn:

- Full directory structure and file names being uploaded (including project paths, customer file names, folder hierarchies)
- FTP remote paths and folder structure on the StreamBIM server
- FTP error messages that could reveal server software, version, or internal configuration details
- File sizes which could leak information about data volume
- Upload timing patterns

The log file is placed in the system temp directory with no file-level access control beyond the standard temp directory permissions. Log files are never rotated or cleaned up.

Because this is opt-in via `args.VerboseDiagnostics`, a default setting that enables it would increase the impact significantly.

What a fix involves

1. **Restrict what is logged**: Replace full file paths with relative paths or anonymized names where the path reveals project structure. Log file counts or hashes instead of full paths and names.
2. **Filter exception messages**: Do not log raw FTP exception messages. Log a canned message like "FTP operation failed: {error category}" instead of the server-returned message that could leak infrastructure details.
3. **Log rotation and cleanup**: Implement log file rotation (max age, max size) and cleanup on upload completion or at application exit.
4. **Consider a standard logging framework**: Replace direct file appends with a structured logging library that supports log levels, rotation, and redaction out of the box.
5. **Review default value**: Ensure `VerboseDiagnostics` defaults to `false` in production configurations so verbose logging is only enabled when needed for troubleshooting.

References

- SEC-001: StreamBIM credential storage and FTP transfer security (same subsystem)
- SEC-002: Dalux API key handling (similar API service area)

SEC-012: Personal data in DaluxUser model with no identified access path

Severity	Info - Personal data fields exist in a model but no code path exposes them to unauthorized access. The data is not in git, not exported, and not served by a web API.
Status	Open
Category	Personal data
Personal data	Email addresses, first names, last names — stored in `DaluxUser` model, no external consumers identified
Location	`src/Assistant/dotnet/DaluxCloudUpload/Models/DaluxModels.cs:634-653` (DaluxUser model definition)
First seen	20261030-034207
Last verified	20261030-034207
Introduced	Not determined

What

The `DaluxUser` class (a DTO for the external Dalux API) contains three personal data fields: `Email`, `FirstName`, and `LastName`. The `DaluxApiService` class provides a `GetProjectUsersAsync` method that fetches a list of `DaluxUser` objects from the Dalux API for a given project ID. This is a CLI extension that calls the Dalux API — it is not a web service that serves data to callers.

Evidence

- Model definition: `src/Assistant/dotnet/DaluxCloudUpload/Models/DaluxModels.cs:634-653`:
  - `Email` (string, line 643)
  - `FirstName` (string?, line 646)
  - `LastName` (string?, line 649)
- API method: `src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs:837`:

```
public async Task<DaluxApiResponse<List<DaluxUser>>> GetProjectUsersAsync(string projectId, CancellationToken [value redacted] = default)
```

- No callers of `GetProjectUsersAsync` exist anywhere in the codebase (confirmed via `grep -rn`).
- `DaluxCloudDownload` (the sibling project) does not reference `DaluxUser`.
- No CSV/Excel export, mail, or log functionality references these fields.

Impact

Personal data (names, email addresses) exists in the model as a DTO for an external API. No code path within this repository exposes it to unauthorized parties:

1. **Who can read it**: The CLI calls the Dalux API to fetch data; there is no inbound HTTP endpoint that serves this data to other users.
2. **Where it leaves the system**: The data is sent TO Dalux (via the API client), not exported from this repo. No mail, export, or log paths identified.
3. **What it is kept for**: No persistence or storage in this repo; data is held transiently during API calls.

However, the `GetProjectUsersAsync` method itself returns `List<DaluxUser>` including all personal data fields. If a caller invokes it, they receive names and emails of all project members. The method should be checked by reviewers to confirm that callers apply appropriate project-level authorization.

References

- security-facts.md section 5: 3 stored fields, 0 file handlers, 0 routes
- SEC-002 (Dalux API key handling) — related to Dalux integration security

Dependencies

tools-extensions-public

Components	Notes
10	7 NuGet packages (direct), 3 build tool dependencies, 20+ app projects (zero-version entries in SBOM)
SBOM file	CycloneDX 1.6, generated by cdxgen 13.0.1

NuGet packages

Package	Version	Direct / Transitive	Notes
FluentFTP	48.0.1	Direct (StreamBIMUploader, StreamBIMDownloader)	FTP client used for file transfers; TLS 1.2 Explicit per SEC-001. Check NuGet versions for latest.
Meziantou.Framework.Win32.CredentialManager	1.7.3	Direct (DaluxCloudUpload, DaluxCloudDownload, StreamBIMUploader, StreamBIMDownloader)	Windows Credential Manager wrapper. Check for updates.
Microsoft.CSharp	4.7.0	Direct (RunCommand, SaveModel)	**Old** - from .NET Core 3.1 era (late 2019). Used only by AutoCAD projects that already have RCE findings (SEC-004, SEC-005). LOW - outdated but not directly exploitable.
System.ComponentModel.Annotations	5.0.0	Direct (OpenDocument, Navisworks)	From .NET 5.0 (2020). Standard library, low risk. Check for updates.
LibGit2Sharp	0.31.0	Direct (ExtensionBuilder, build only)	Build tool only - git operations for changed-project detection. Check for updates.
NuGet.Frameworks	7.9.0	Direct (ExtensionBuilder, build only)	Build tool only. Check for updates.
Nuke.Common	10.1.0	Direct (ExtensionBuilder, build only)	Build orchestration framework. Check for updates.

> **Note:** This page lists direct dependencies from manifests and the SBOM. Transitive dependencies are not visible in the SBOM (the generator does not recurse into them). The reviewer should verify transitive deps through `dotnet list dependency --include-transitive` for each project. Version X in this table means "check advisories" - no CVE claims are made here.

Software bill of materials

One CycloneDX file per source repo, generated by the agent runner before every security review from the manifests in the checkout (OWASP cdxgen, spec 1.6). See SBOM/README.md for the index and how to regenerate. Dependencies.md is written from these files.

Source-only: versions as declared in the manifests, no license lookup, no vulnerability data. Serial numbers and timestamps are stripped so a file changes only when a dependency changes; git history is the change log. The same generator command is meant to run in the build pipeline so a released installer ships the SBOM it was built from.

Software bill of materials

One CycloneDX 1.6 file per source repo, generated by the runner (`services/aec.agents/sbom/run-sbom.sh`, OWASP cdxgen 13.0.1) from the manifests in the checkout before each security review. Source-only: versions as declared, no license lookup, no vulnerability data. The files are the authority for `Dependencies.md`; the agent reads them and never edits them. Serial numbers and timestamps are stripped so a file only changes when a dependency changes.

Repo	File	Components	Ecosystems
tools-extensions-public	tools-extensions-public.cdx.json	10	github 3, nuget 7

Regenerate by hand with `services/aec.agents/sbom/run-sbom.sh <workspace> <output-dir>`, which pins the exact cdxgen image by digest. The same command belongs in the build pipeline so a release ships the SBOM it was built from.

Talk to the security agent

This folder is written by the AEC security agent: it reviews this repository for security weaknesses and writes each one up with evidence; it never fixes anything. It runs at night, and only when something changed - and an edit to this file counts. Write here to steer it or ask it something; you will find the answer in this file after the next run.

How to edit. If a pull request from the agent is open, edit this file on its branch (`security/ai-security`) - no review needed. Otherwise edit it on the default branch the way you normally would. Add your text under one of the three headings below. Do not edit the lines the agent writes (IDs, Status, Answer), except to close an item; add a new line under an answer to follow up.

Rules shared by every repo are kept in `AEC.Nvidia.LLM/services/aec.agents/memory/` and reach this agent on every run; a line under Standing instructions below outranks them for this repo.

Standing instructions

Rules the agent follows on every run, one per line. Only people write here.

- Report only. Never change code, config or pipelines. Describe what a fix involves; do not write it.
- Never copy a secret value or personal data into any page. Location and kind only.
- Export folders (`*Exports/`) hold customer data files: report their presence in git as a data-protection finding once; do not enumerate their contents.

Requests

One-off work. Start each with a level-3 heading (`### What you want`) and describe it below. The agent gives it an ID and marks it Done, In progress or Declined, with a reason.

Questions and statements

Ask a question, or state something you believe and the agent checks it. Start each with a level-3 heading (`### Your question`). The agent answers under it, with evidence. When you are done, change the word after `Status:` on the item's own status line to `Closed`; closed items move to `.agent/conversation-archive.md`.